

Deep Learning Project Report

Cindy Nguyen

May 3, 2024

1. Overview and Goals

During Spring 2024, I worked on a deep learning project involving the MIMIC-IV medical dataset, specifically the electrocardiogram (ECG) and the left ventricular ejection fraction (LVEF). My goal was to use deep learning models to predict the LVEF classification of patients given their 10-second ECG segments. First, I consolidated the number of ECGs to be used, and labeled them with their LVEF by searching in the corresponding text. I divided up the LVEF into 5 categories: Hyperdynamic, Normal, Mildly depressed, Depressed, and Severely depressed. Based on the resulting dataframe, I retrieved the selected ECGs in a numpy array from the MIMIC-IV database. Then, I adapted the baseline model and AlexNet model to the parameters of this new dataset. My LVEF prediction results using 1 of the 12 ECG leads provided are plotted as ROC curves and displayed in confusion matrices.

2. MySQL_getLVEF_text

The first of 3 scripts I worked on this semester is the `MySQL_getLVEF_text`, in which I labeled the ECGs with their LVEF noted in the text. In order to consolidate the number of ECGs to be used in the model, I selected ECGs based on filtering the time difference between the `ecg_time` and `admittime` to be less than 1 month (30 days). Also, I only kept the first ECG entry per `hadm_id` since there were multiple ECGs with the same `hadm_id`. My process is shown in the following code snippets.

[illegible]

```
# Drop the 'time_difference' column along with others not needed
merged_df = merged_df.drop(columns=['time_difference', 'note_type', 'charttime', 'storetime'])
merged_df.reset_index(drop=True, inplace=True)
merged_df
```

46424 rows x 11 columns

[illegible]

```
[25]: "\nlvef_matches_df = merged_df['text'].str.extractall(pattern)\nfor test_str in lvef_matches_df\n    nany(chr.isdigit() for chr in test_str)\nmerged_df['extracted_text'] = \"n\"
```

[27]:	merged df												
	note_id	subject_id	hadm_id	note_seq	text	admittime	disctime	study_id	file_name	ecg_time	path	extracted_text	extracted_text2
0	10000764-DS-11	10000764	27897940	11	vnName: ____ Unit No: ____ vn ____	2132-10-14 23:33.00	2132-10-19 16:30.00	47218930	47218930	2132-10-14 19:40.00	files/p1000/p10000764/47218930	= 55 vn%	55.0
1	10000826-DS-11	10000826	20032325	17	vnName: ____ Unit No: ____ vn ____	2146-12-05 19:07.00	2146-12-12 16:30.00	40695233	40695233	2146-12-12 08:42.00	files/p1000/p10000826/40695233	>55%.	55.0
2	10000980-DS-20	10000980	29654838	20	vnName: ____ Unit No: ____ vn ____	2188-01-03 17:43.00	2188-01-05 17:30.00	42546971	42546971	2188-01-03 12:33.00	files/p1000/p10000980/42546971	>55%.	55.0
3	10000980-DS-21	10000980	26913865	21	vnName: ____ Unit No: ____ vn ____	2189-06-27 07:38.00	2189-07-03 03:00.00	48376834	48376834	2189-06-27 06:43.00	files/p1000/p10000980/48376834	= vn35-4	35.0
4	10000980-DS-26	10000980	20897796	26	vnName: ____ Unit No: ____ vn ____	2193-08-15 01:01.00	2193-08-17 15:07.00	48778222	48778222	2193-08-14 23:13.00	files/p1000/p10000980/48778222	50% ____	50.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
46419	19998330-DS-16	19998330	27282608	16	vnName: ____ Unit No: ____	2177-07-25 04:34.00	2177-07-26 14:07.00	42200360	42200360	2177-07-25 03:25.00	files/p1999/p19998330/42200360	on ____	NaN
46420	19998591-DS-8	19998591	24349193	8	vnName: ____ No: ____ vn ____	2185-07-03 20:20.00	2185-08-03 14:42.00	42337063	42337063	2185-07-03 17:54.00	files/p1999/p19998591/42337063	>55%.	55.0
46421	19998784-DS-10	19998784	25175148	10	vnName: ____ Unit No: ____	2119-08-11 11:36.00	2119-08-11 11:55.00	47639679	47639679	2119-07-25 05:58.00	files/p1999/p19998784/47639679	55% vn1	55.0
46422	19998840-DS-20	19998840	26071774	20	vnName: ____ Unit No: ____	2164-07-25 02:27.00	2164-07-28 12:15.00	46519385	46519385	2164-07-24 22:31.00	files/p1999/p19998840/46519385	>55%.	55.0
46423	19999587-DS-2	19999587	23865745	2	vnName: ____ Unit No: ____	2145-11-02 21:38.00	2145-11-11 12:57.00	41190887	41190887	2145-11-02 19:54.00	files/p1999/p19999587/41190887	55% vn	55.0

To examine the distribution of LVEF values, I checked the counts of distinct numerical values. Since some phrases did not have numbers in them, there were about 4500 NaNs, which I would like to minimize as much as possible.

```
[28]: def check_NaN_counts_in_LVEF_extracted_text():
    value_counts = merged_df['extracted_text2'].value_counts(dropna=False)

    # Convert the value counts to a dictionary or DataFrame if needed
    value_counts_dict = value_counts.to_dict()
    value_counts_df = value_counts.reset_index()

    # Print or use the dictionary or DataFrame as needed
    print(value_counts_dict)
    print(value_counts_df)
    check_NaN_counts_in_LVEF_extracted_text()

#merged_df[merged_df['extracted_text2'] == '685'] ['text']
[55.0: 19913, nan: 4570, 50.0: 3403, 40.0: 2368, 45.0: 2250, 60.0: 1911, 35.0: 1045, 65.0: 1431, 70.0: 1382, 30.0: 1163, 20.0: 936, 25.0: 860, 75.0: 522, 15.0: 333, 61.0: 163, 62.0: 154, 63.0: 136, 58.0: 135, 10.0: 132, 67.0: 129, 64.0: 124, 68.0: 122, 57.0: 112, 66.0: 107, 59.0: 104, 56.0: 90, 49.0: 86, 48.0: 76, 69.0: 76, 47.0: 74, 43.0: 72, 54.0: 70, 71.0: 69, 53.0: 67, 52.0: 66, 38.0: 65, 42.0: 62, 44.0: 62, 41.0: 60, 51.0: 60, 39.0: 60, 46.0: 54, 34.0: 54, 32.0: 53, 36.0: 52, 72.0: 52, 31.0: 46, 74.0: 45, 29.0: 45, 27.0: 44, 23.0: 40, 28.0: 40, 37.0: 40, 33.0: 39, 73.0: 39, 80.0: 33, 26.0: 33, 22.0: 32, 19.0: 28, 24.0: 28, 3.0: 26, 76.0: 24, 17.0: 24, 77.0: 20, 4.0: 19, 18.0: 17, 2.0: 14, 6.0: 12, 21.0: 12, 5.0: 11, 79.0: 11, 12.0: 11, 85.0: 10, 1.0: 10, 78.0: 7, 82.0: 7, 14.0: 6, 13.0: 6, 0.0: 6, 81.0: 4, 8.0: 4, 83.0: 4, 7.0: 4, 11.0: 3, 86.0: 3, 84.0: 2, 16.0: 2, 88.0: 2, 685.0: 1, 740.0: 1, 505.0: 1, 9.0: 1, 3040.0: 1, 90.0: 1]
index extracted_text2
0 55.0 19913
1 NaN 4570
2 50.0 3403
3 40.0 2368
4 45.0 2250
.. ..
89 740.0 1
90 505.0 1
91 9.0 1
92 3040.0 1
93 90.0 1

[94 rows x 2 columns]
```

In order to decrease the NaN counts, I tried another approach of finding keywords corresponding to each of the LVEF classification categories. If the keywords of one category were found, I estimated the LVEF to be the numerical average of the LVEF range of that category. For example, if the keyword “normal LVEF” was found, the LVEF value would get the number 55.0 since the normal LVEF range is between 50 and 60.

```
[29]: #Still a lot of NaN... I will keep processing the text
import re
#second round, now for replacing the mild, severe sentences with numbers
pattern1 = r'severely depressed LVEF|severely depressed EF|severely depressed|everely depressed|everely\ndepressed|systolic function is severely depressed'
pattern2 = r'mildly depressed|mildly\ndepressed'
pattern3 = r'epressed LVEF|moderately depressed|epressed\nLVEF'
pattern4 = r'normal LVEF|normal\nLVEF|reserved LVEF|normal EF'
replacement1 = 25.0 #SEV
replacement2 = 35.0 #DEP
replacement3 = 45.0 #MILD
replacement4 = 55.0 #NORM
replacement5 = r'(?!\d)LVEF|ejction fraction|(.{4})'

# Select rows with NaN values in 'extracted_text' column
nan_rows = merged_df['extracted_text2'].isna()

# Apply conditional replacements on the selected rows
merged_df.loc[nan_rows, 'extracted_text2'] = merged_df.loc[nan_rows, 'text'].apply(lambda x: replacement1 if re.search(pattern1, x) else (replacement2 if re.search(pattern2, x) else (replacement3 if re.search(pattern3, x) else (replacement4 if re.search(pattern4, x) else replacement5)))
#Checking again if the NaN count has decreased
check_NaN_counts_in_LVEF_extracted_text()

[55.0: 20390, 50.0: 3403, nan: 2987, 45.0: 2562, 40.0: 2368, 35.0: 2009, 60.0: 1911, 25.0: 1490, 65.0: 1431, 70.0: 1382, 30.0: 1163, 20.0: 936, 75.0: 522, 15.0: 333, 61.0: 163, 62.0: 154, 63.0: 136, 58.0: 135, 10.0: 132, 67.0: 129, 64.0: 124, 68.0: 122, 57.0: 112, 66.0: 107, 59.0: 104, 56.0: 90, 49.0: 86, 48.0: 76, 69.0: 76, 47.0: 74, 43.0: 72, 54.0: 70, 71.0: 69, 53.0: 67, 52.0: 66, 38.0: 65, 42.0: 62, 44.0: 62, 41.0: 60, 51.0: 60, 39.0: 60, 46.0: 54, 34.0: 54, 32.0: 53, 36.0: 52, 72.0: 52, 31.0: 46, 74.0: 45, 29.0: 45, 27.0: 44, 23.0: 40, 28.0: 40, 37.0: 40, 33.0: 39, 73.0: 39, 80.0: 33, 26.0: 33, 22.0: 32, 19.0: 28, 24.0: 28, 3.0: 26, 76.0: 24, 17.0: 24, 77.0: 20, 4.0: 19, 18.0: 17, 2.0: 14, 6.0: 12, 21.0: 12, 5.0: 11, 79.0: 11, 12.0: 11, 85.0: 10, 1.0: 10, 78.0: 7, 82.0: 7, 14.0: 6, 13.0: 6, 0.0: 6, 81.0: 4, 8.0: 4, 83.0: 4, 7.0: 4, 11.0: 3, 86.0: 3, 84.0: 2, 16.0: 2, 88.0: 2, 685.0: 1, 740.0: 1, 505.0: 1, 9.0: 1, 3040.0: 1, 90.0: 1]
index extracted_text2
0 55.0 20390
1 50.0 3403
2 NaN 2987
3 45.0 2562
4 40.0 2368
.. ..
89 740.0 1
90 505.0 1
91 9.0 1
92 3040.0 1
93 90.0 1

[94 rows x 2 columns]
```

Afterwards, I counted the number of entries in each of the 5 LVEF categories. Out of a total of 43433 entries, the distribution shows that 855 is Hyperdynamic (class 0), 30152 is Normal (class 1), 5476 is Mildly depressed (class 2), 3581 is Depressed (class 3), 3369 is Severely depressed (class 4). The overwhelming majority in the dataset is Normal, which is completely expected but we just might need to take this into consideration when running the deep learning models.

```
[35]: #Now filtering, first I will create a copy then erase the rows that had the NOT FOUND unresolved,
#and the ones that have the time difference too big
dataset_df = merged_df.copy()

# Now, 'dataset_df' will contain only the rows where 'extracted_text' is not 'NOT FOUND'
dataset_df = dataset_df[dataset_df['extracted_text2'].notna()]
dataset_df = dataset_df[dataset_df['category'] == 0]

# Resetting the Index
dataset_df.reset_index(drop=True, inplace=True)
```

```
[36]: dataset_df
```

	note_id	subject_id	hadm_id	note_seq	text	admittime	disctime	study_id	file_name	ecg_time	path	extracted_text	extracted_text2	category
0	10000764-DS-11	10000764	27897940	11	lvName: ____ Unit No: ____\n\n	2132-10-14 23:31.00	2132-10-19 16:30.00	47218930	47218930	2132-10-14 19:40.00	file:sp1000p10000764\n47218930\47218930	= 55 v%	55.0	1.0
1	10000826-DS-17	10000826	20032235	17	lvName: ____ Unit No: ____\n\nlvAdm: ____	2146-12-05 19:07.00	2146-12-12 16:30.00	40695233	40695233	2146-12-12 08:42.00	file:sp1000p10000826\n40695233\40695233	>55%.	55.0	1.0
2	10000980-DS-20	10000980	29654838	20	lvName: ____ Unit No: ____\n\nlvAdm: ____	2189-01-03 17:41.00	2189-01-05 17:30.00	42546971	42546971	2189-01-03 12:33.00	file:sp1000p10000980\n42546971\42546971	>55%.	55.0	1.0
3	10000980-DS-21	10000980	26913865	21	lvName: ____ Unit No: ____\n\nlvAdm: ____	2189-06-27 07:38.00	2189-07-03 03:00.00	48376834	48376834	2189-06-27 06:43.00	file:sp1000p10000980\n48376834\48376834	= ln35-4	35.0	3.0
4	10000980-DS-26	10000980	20897796	26	lvName: ____ Unit No: ____\n\nlvAdm: ____	2193-08-15 01:01.00	2193-08-17 15:07.00	48778222	48778222	2193-08-14 23:13.00	file:sp1000p10000980\n48778222\48778222	50% ____	50.0	1.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
43428	19998330-DS-16	19998330	27282608	16	lvName: ____ Unit No: ____	2177-07-25 04:34.00	2177-07-26 14:07.00	42200360	42200360	2177-07-25 03:25.00	file:sp1999p19998330\n42200360\42200360	on ____	55.0	1.0
43429	19998591-DS-8	19998591	24349193	8	lvName: ____ No: ____\n\n	2185-07-03 20:20.00	2185-08-03 14:42.00	42337063	42337063	2185-07-03 17:54.00	file:sp1999p19998591\n42337063\42337063	>55%.	55.0	1.0
43430	19999784-DS-10	19999784	25715748	10	lvName: ____ Unit No: ____	2119-08-11 11:36.00	2119-08-19 11:55.00	47639679	47639679	2119-07-25 05:58.00	file:sp1999p19999784\n47639679\47639679	55%\n(1	55.0	1.0
43431	19999840-DS-20	19999840	26071774	20	lvName: ____ Unit No: ____\n	2164-07-25 00:27.00	2164-07-28 12:15.00	46519385	46519385	2164-07-24 22:31.00	file:sp1999p19999840\n46519385\46519385	>55%.	55.0	1.0
43432	19999987-DS-2	19999987	23865745	2	lvName: ____ Unit No: ____\n	2145-11-02 21:38.00	2145-11-11 12:57.00	41190887	41190887	2145-11-02 19:54.00	file:sp1999p19999987\n41190887\41190887	55%\n	55.0	1.0

43433 rows x 14 columns

```
[30]: #print(merged_df['extracted_text2'])
hyper = (merged_df['extracted_text2']).between(71,100,'both').sum()
norm = (merged_df['extracted_text2']).between(50,70,'both').sum()
mild = (merged_df['extracted_text2']).between(40,49,'both').sum()
dep = (merged_df['extracted_text2']).between(30,39,'both').sum()
sev = (merged_df['extracted_text2']).between(0,29,'both').sum()
#remove = (merged_df['extracted_text2'] > 100).sum()
categs_df = pd.DataFrame(data={"categories": [hyper, norm, mild, dep, sev]})
print(categs_df)
```

```
categories
0      855
1    30152
2     5476
3     3581
4     3369
```

```
[31]: merged_df.loc[merged_df['extracted_text2'].between(71,100,'both'),'category'] = 0
merged_df.loc[merged_df['extracted_text2'].between(50,70,'both'),'category'] = 1
merged_df.loc[merged_df['extracted_text2'].between(40,49,'both'),'category'] = 2
merged_df.loc[merged_df['extracted_text2'].between(30,39,'both'),'category'] = 3
merged_df.loc[merged_df['extracted_text2'].between(0,29,'both'),'category'] = 4
merged_df.loc[merged_df['extracted_text2'] > 100,'category'] = -1
```

I generated a dataframe of the selected ECG file paths and their corresponding LVEF category to be able to retrieve the ECGs we need in the next step.

note_seq	text	admittime	disctime	study_id	file_name	ecg_time	path	extracted_text	extracted_text2	category
1	11 Name: ____	2132-10-14 23:31.00	2132-10-19 16:30.00	47218930	47218930	2132-10-14 19:40.00	file:sp1000p10000764\n47218930\47218930	= 55 %	55.0	1.0
2	17 Name: ____	2146-12-05 19:07.00	2146-12-12 16:30.00	40695233	40695233	2146-12-12 08:42.00	file:sp1000p10000826\n40695233\40695233	>55%.	55.0	1.0
3	20 Name: ____	2188-01-03 17:41.00	2188-01-05 17:30.00	42546971	42546971	2188-01-03 12:33.00	file:sp1000p10000980\n42546971\42546971	>55%.	55.0	1.0
4	21 Name: ____	2189-06-27 07:38.00	2189-07-03 03:00.00	48376834	48376834	2189-06-27 06:43.00	file:sp1000p10000980\n48376834\48376834	= 35-4	35.0	3.0
5	26 Name: ____	2193-08-15 01:01.00	2193-08-17 15:07.00	48778222	48778222	2193-08-14 23:13.00	file:sp1000p10000980\n48778222\48778222	50% ____	50.0	1.0
6	19 Name: ____	2131-07-30 21:40.00	2131-08-04 14:10.00	49368993	49368993	2131-07-30 13:04.00	file:sp1000p100010001\n49368993\49368993	>55%.	55.0	1.0
7	20 Name: ____	2150-11-21 23:02.00	2150-11-23 16:46.00	46566322	46566322	2150-11-21 18:48.00	file:sp1000p100010001\n46566322\46566322	55%\n	55.0	1.0
8	30 Name: ____	2130-04-15 21:10.00	2130-04-19 16:00.00	45070027	45070027	2130-04-08 16:05.00	file:sp1000p100010001\n45070027\45070027	>55%.	55.0	1.0
9	12 Name: ____	2185-11-23 08:19.00	2185-11-26 15:40.00	44466869	44466869	2185-11-22 15:49.00	file:sp1000p100020002\n44466869\44466869	55%\n D	55.0	1.0
10	7 Name: ____	2160-05-18 07:45.00	2160-05-23 13:30.00	45357652	45357652	2160-04-24 10:53.00	file:sp1000p100020002\n45357652\45357652	55%.	55.0	1.0
11	13 Name: ____	2141-05-22 20:17.00	2141-05-29 17:41.00	43920149	43920149	2141-05-22 19:42.00	file:sp1000p100020002\n43920149\43920149	= 7 35-	35.0	3.0
12	3 Name: ____	2145-03-12 23:02.00	2145-03-18 17:50.00	41514455	41514455	2145-02-25 10:07.00	file:sp1000p100020002\n41514455\41514455	70%.	70.0	1.0
13	18 Name: ____	2174-09-16 17:14.00	2174-09-21 16:07.00	40064176	40064176	2174-09-16 14:37.00	file:sp1000p100030003\n40064176\40064176	>55%.	55.0	1.0
14	22 Name: ____	2175-10-08 13:56.00	2175-10-17 17:30.00	41113602	41113602	2175-10-08 13:23.00	file:sp1000p100030003\n41113602\41113602	>55%.	55.0	1.0
15	26 Name: ____	2176-01-06 15:52.00	2176-01-14 18:09.00	47468582	47468582	2176-01-06 11:32.00	file:sp1000p100030003\n47468582\47468582	>55%.	55.0	1.0
16	6 Name: ____	2179-06-30 16:35.00	2179-07-02 17:46.00	47053082	47053082	2179-06-30 09:28.00	file:sp1000p100030003\n47053082\47053082	55%\n wit	55.0	1.0
17	19 Name: ____	2137-08-04 00:07.00	2137-09-02 17:05.00	42107057	42107057	2137-08-18 07:58.00	file:sp1000p100030003\n42107057\42107057	50-55%.	50.0	1.0
18	7 Name: ____	2166-02-15 11:06.00	2166-02-19 16:02.00	47886467	47886467	2166-02-15 10:26.00	file:sp1000p100030003\n47886467\47886467	60%\n in	60.0	1.0
19	8 Name: ____	2166-02-15 13:35.00	2166-02-21 17:42.00	47886467	47886467	2166-02-15 10:26.00	file:sp1000p100030003\n47886467\47886467	60%\n in	60.0	1.0
20	19 Name: ____	2146-02-18 16:49.00	2146-02-19 20:00.00	49493393	49493393	2146-02-22 11:28.00	file:sp1000p100030003\n49493393\49493393	27%\n S	27.0	4.0

3. RetrieveECGs

The second script I worked on is RetrieveECGs. Due to the large number of ECGs in the dataframe (about 40000), Titan's RAM unfortunately could not handle saving all the ECGs

at once so I had to save the ECGs into 3 numpy arrays. I also had to downsample them from 500Hz to 100Hz to avoid this issue. Then, I merged these 3 arrays back into 1 large numpy array of dimension (43433, 1000, 12) to be used in the next step for training and testing the deep learning models.

```
[1]: import numpy as np
import wfdb as wfdb
import pandas as pd
from pathlib import Path

11: #load df_MCS patient records from a csv file
df_MCS_patient_records = pd.read_csv('MIMICIV_MCS_patient_records-Cindy.csv', skiprows=(lambda x: x > 0 and x < 30000))
ecg_path = '/mnt/data3/mimiciv_ecg/physionet.org/files/mimic-iv-ecg/1.0'

[3]: #all ECGs from dataframe.shape

12: #Example. Get the ECG from the paths in the dataframe
def retrieve_ECGs_from_dataframe_list(ecgs_main_path, df_with_ecg_paths, column_name_of_path):
    for index, row in df_with_ecg_paths.iterrows():
        filename = row[column_name_of_path]
        record = wfdb.rdrecord(str(Path(ecgs_main_path) / filename))
        retrieved_ECG = record.p_signal[:, :] # Keep the original sampling rate
        if index % 500 == 0:
            print(f"ECG for patient {index} is of shape {retrieved_ECG.shape}") # Uncomment if you want to debug
        if index == 0:
            #all ECGs = np.zeros((1, retrieved_ECG.shape[0], retrieved_ECG.shape[1]))
            all_ECGs = np.zeros((len(df_MCS_patient_records), int(retrieved_ECG.shape[0]/5), retrieved_ECG.shape[1]))
            all_ECGs[0, :, :] = retrieved_ECG[:, 5, :]

        else:
            #retrieved ECG = np.expand_dims(retrieved_ECG, axis=0)
            #all ECGs = np.vstack((all_ECGs, retrieved_ECG))
            #Hi Cindy, I think that instead of this couple of sentences you could use
            all_ECGs[index, :, :] = retrieved_ECG[:, 5, :] #, because now that I see it we are stacking every time the vector
            #And that might take longer time
            # I just tried this but it threw me this error:
            #IndexError: index 1 is out of bounds for axis 0 with size 1

    print(f'Finished retrieving {index+1} ECGs. Shape of all the ecgs: {all_ECGs.shape}')
    return all_ECGs

all_ECGs_from_dataframe = retrieve_ECGs_from_dataframe_list(ecg_path, df_MCS_patient_records, 'path')

np.save('nparray3_ECGs', all_ECGs_from_dataframe)

ECG for patient 0 is of shape (5000, 12)
ECG for patient 500 is of shape (5000, 12)
ECG for patient 1000 is of shape (5000, 12)
ECG for patient 1500 is of shape (5000, 12)
ECG for patient 2000 is of shape (5000, 12)
ECG for patient 2500 is of shape (5000, 12)
.....

[14]: a1 = np.load('nparray1_ECGs.npy')
a2 = np.load('nparray2_ECGs.npy')
a3 = np.load('nparray3_ECGs.npy')
a = np.concatenate((a1, a2[1:], a3[1:]), axis=0)
#a = np.vstack((a1, a2[1:], a3[1:]))
print(a.shape)
np.save('nparrayAll_ECGs', a)

(43433, 1000, 12)
```

4. MIMIC_IV_LVEF_Tests

In the script MIMIC_IV_LVEF_Tests, I used deep learning models to predict the LVEF classification of patients given 1 of the 12 ECG leads. First, I adjusted the parameters of the previous models used on MIMIC-III to fit the new MIMIC-IV dataset, such as the dimension and the number of layers. Since I realized the ECGs contained some NaNs that interfered with the prediction accuracy, I replaced all NaNs with the average value of its row. I also one-hot encoded the Y data of LVEF classification from 1 column with entries 0-4 (corresponding to the 5 categories) to 5 columns with binary entries 0-1.

```

for i in range(43433):
    X[i,:] = np.nan_to_num(X1[i,:],nan=np.nanmean(X1[i,:]))
Y = pd.get_dummies(Y1, columns=['category', ])
Y = Y.to_numpy()

print(X.shape)
print(Y.shape)
print(X[0,:])
print(Y[:10])

(43433, 1000)

(43433, 5)

[-0.055 -0.035 -0.035 -0.03 -0.035 -0.015 -0.025 -0.015 -0.025 -0.015
 0.025 0.04 0.08 0.055 0.045 0.01 -0.03 -0.045 -0.055 -0.07
-0.025 -0.03 -0.035 -0.055 -0.055 -0.05 -0.065 -0.045 -0.045 -0.05

```

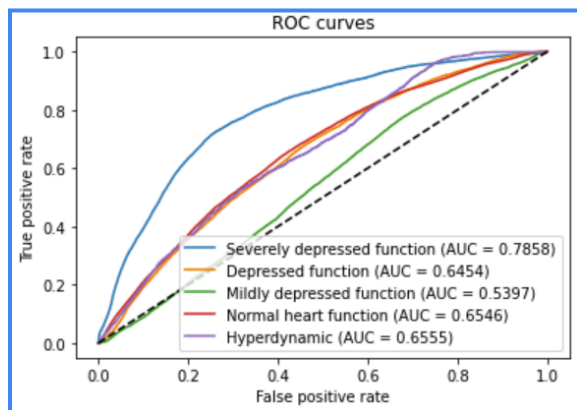
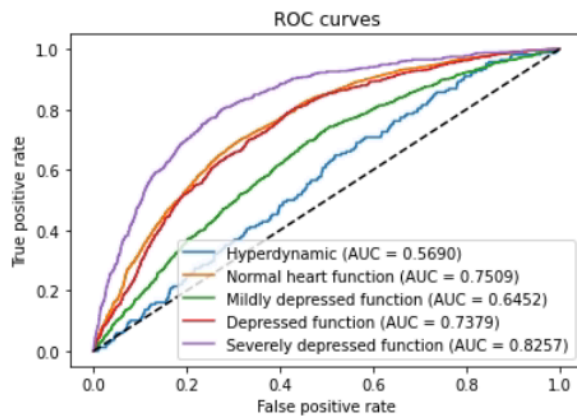
Below are the new ROC curves that I plotted after running the baseline models, compared to the old ROC curves on MIMIC-III data (blue borders). Except for Hyperdynamic, all the other categories had slightly higher AUC scores than the old ROC curves. Hyperdynamic having a lower score might be due to not having enough ECG data labeled in this category (we only had 855 entries). I also displayed the new results in a confusion matrix and compared it to the old confusion matrix on MIMIC-III data (blue borders).

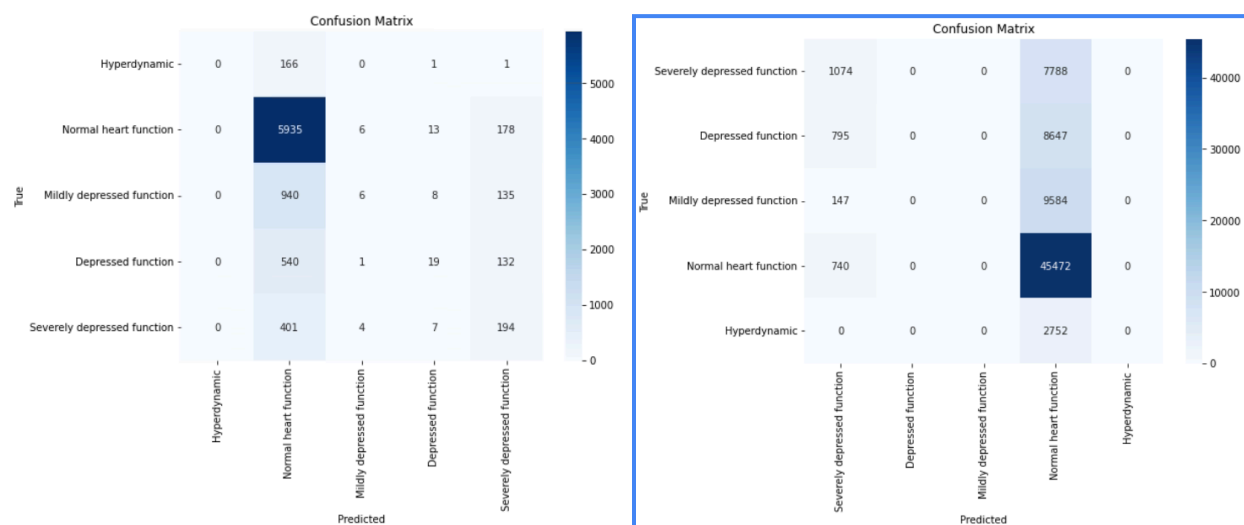
```

[76]: # predict on test set
y_pred = model.predict(X_test)
plot_ROC_curves (n_classes, class_names, Y_test, y_pred)
conf_matrix = get_and_plot_confusion_matrix (class_names, Y_test, y_pred)

272/272 [=====] - 2s 7ms/step

```





According to the following new and old classification reports, the precision and accuracy parameters were slightly better in MIMIC-IV than in MIMIC-III.

```
#print the classification report
print(classification_report(np.argmax(Y_test, axis=1), np.argmax(y_pred, axis=1), target_names=class_names))
```

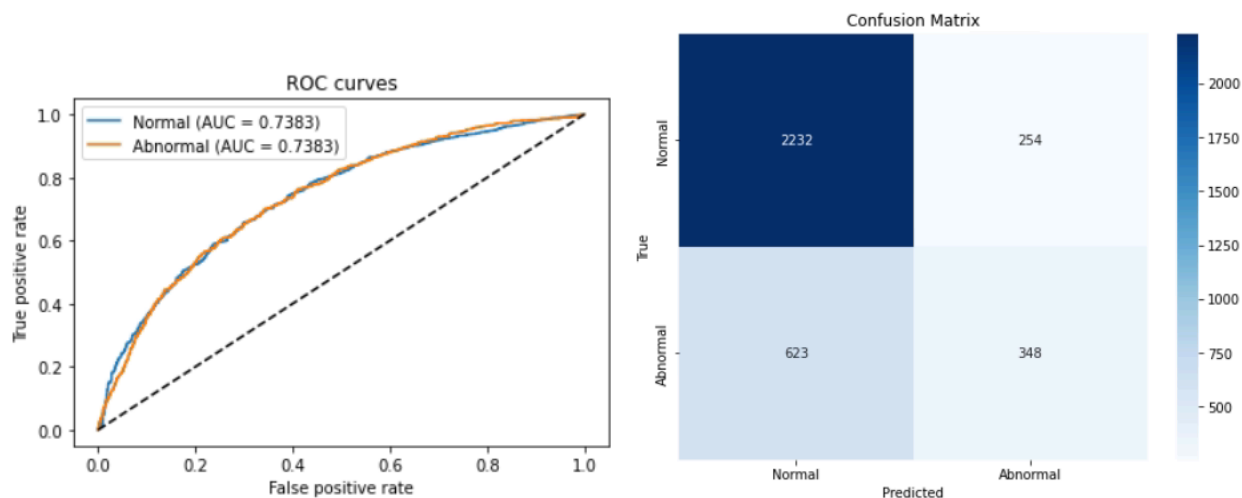
	precision	recall	f1-score	support
Hyperdynamic	0.00	0.00	0.00	168
Normal heart function	0.74	0.97	0.84	6132
Mildly depressed function	0.35	0.01	0.01	1089
Depressed function	0.40	0.03	0.05	692
Severely depressed function	0.30	0.32	0.31	606
accuracy			0.71	8687
macro avg	0.36	0.26	0.24	8687
weighted avg	0.62	0.71	0.62	8687

	precision	recall	f1-score	support
Severely depressed function	0.39	0.12	0.18	8862
Depressed function	0.00	0.00	0.00	9442
Mildly depressed function	0.00	0.00	0.00	9731
Normal heart function	0.61	0.98	0.76	46212
Hyperdynamic	0.00	0.00	0.00	2752
accuracy			0.60	76999
macro avg	0.20	0.22	0.19	76999
weighted avg	0.41	0.60	0.47	76999

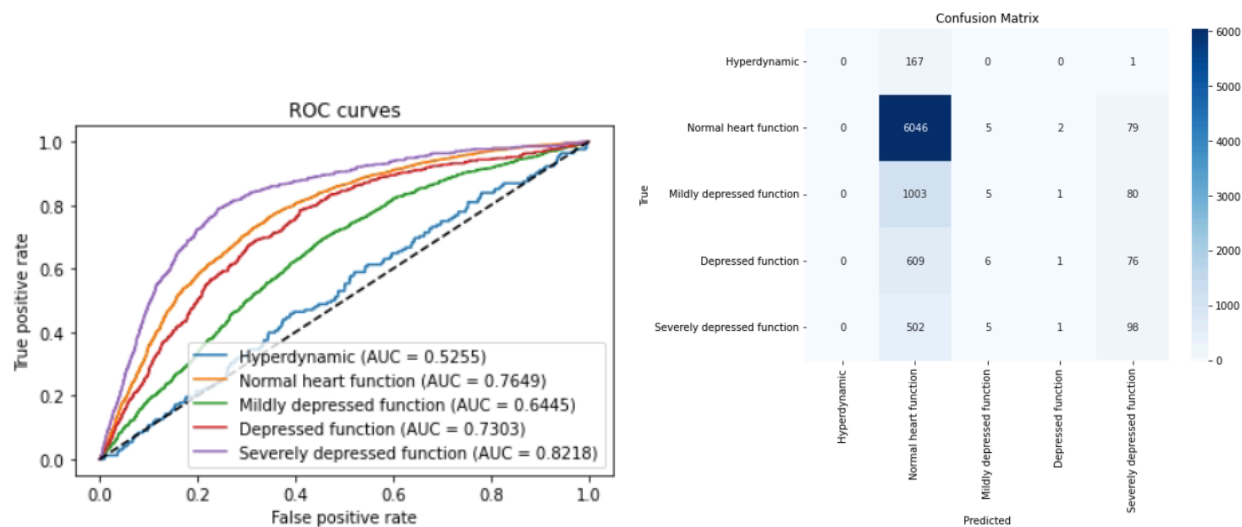
5. Work in Progress and Future Goals

I am currently working on expanding the above deep learning models to go from taking in a 2D input array (from 1 ECG lead) to a 3D one in order to predict from all 12 ECG leads. First, I tried to classify only 2 labels: Normal (normal/hyperdynamic) vs Abnormal (mildly/depressed/severely). My initial intention was to predict only 1 class and evaluate the accuracy of the results.

Testing this approach on the 1-lead data first, I ran into trouble displaying the results from only 1 class, so I decided to separate the 1 class into 2 classes for easier evaluation. The following ROC curves and confusion matrix show the results for 1 lead and 2 classes, with an AUC score of 0.7383.



Next, I tested the original 5-class approach on the 12-lead data to make sure it is working. The results for 12 leads and 5 classes are shown in the ROC curves and confusion matrix below. The low AUC score for Hyperdynamic might be due to the unbalanced dataset as seen in almost all the Hyperdynamic ECGs being predicted as Normal by the model.



Therefore in implementing the new 2-class approach on the 12-lead data, I had to balance the distribution of the dataset by sampling only 4000 of the ECGs classified as normal LVEF, which was originally the majority. However, I have been experiencing some problems with the kernel always dying when I try to concatenate the selected ECGs in 3 dimensions back into the training and testing X datasets. I suspect there might not be enough RAM for the concatenate function to handle merging the amount of data from all 12 leads. As a next step, I will keep working on fixing this issue to be able to obtain viewable results.

For some future goals, I am currently trying out a new function that builds a model implementing the 12 “parallel” convolutional blocks for the full 12-lead ECG data. I will be testing to see how this change in the model’s structure impacts the accuracy of the prediction results. Also, I would like to examine the progression of the ECG and LVEF over time for each patient who got an LVAD to analyze if there are significant improvements after compared to before the LVAD.